

Design Doc: mobile-trip-reconcile-01

1. What this task is

A real ARM64 Android emulator (KVM-accelerated, booted inside the Harbor container itself) with a raw screenshot/tap/swipe/type action space -- no accessibility tree, no "list open apps" or "open app by name" shortcut, no text description of screen contents. The agent must plan a Denver trip (one flight + one hotel, booked together on a Chrome-hosted site called TripDesk) that matches what a contact, Sam, actually needs: the right dates and a combined price under budget. Requirements are never handed to the agent directly -- it has to gather them itself from a real native Calendar app, a real native Contacts app, and a Chat thread (also Chrome-hosted), the way a person actually would before booking something.

Partway through the mission, a corrected message arrives -- as a genuine Android system notification, timed off the agent's own first real action, not off a fixed clock -- that overturns the return-date conclusion the information available at $t=0$ reasonably supported. The deliverable is a confirmed TripDesk booking; grading reads TripDesk's own SQLite database directly, never an LLM judge or a written report.

2. Why this design (methodology lineage)

Built after reviewing MobileWorld and AndroidWorld's published methodology (real device state, backend-DB verification, single-app saturation as their acknowledged limitation) -- their benchmarks themselves are not reused, copied, or derived from anywhere here; no scenario, asset, app, or task text originates from either project. What's borrowed is method, not content:

- **Backend-DB verification**, not string-matching a report or an LLM-as-judge (mirrors MobileWorld).
- **Real device state** (actual content providers, actual first-run dialogs, actual QEMU NAT quirks) instead of a mocked/scripted UI, so the failure modes a model hits are the same ones it would hit on a real phone, not artifacts of a fake harness.
- **Deliberately cross-app**, addressing the single-app-saturation gap both benchmarks flag: Calendar -> Contacts -> Chrome/Chat -> Chrome/TripDesk, switching apps only via the real home screen, the way a human actually would.

3. The core capability under test

Not "can the model book a flight" (that's a solved, saturating capability by itself). The target capability is: **does the agent's final action reflect currently-true state, or a stale first pass that was never re-verified** -- i.e. genuine long-horizon state tracking under a mid-mission

update, not just multi-step tool use. A model that reads once at $t=0$ and commits will look competent right up until the moment the world changes under it.

4. The designed traps

All of these are real, present in the shipped content, not hypothetical:

1. **The core timing trap.** At $t=0$ the SMS thread + Calendar (board meeting Oct 19, 5-6pm, doesn't conflict with a same-day return) genuinely and non-britly support returning ON the 19th. This is explicitly *not* a trick reading -- it's the reasonable conclusion from real information. A corrected message ("actually need to be back by the 18th") lands `delay_seconds` after the agent's first logged action, as a real notification. An agent that books off its first read alone will book the wrong return date.
2. **Anchoring reinforcement.** The pre-correction SMS explicitly ties the 19th to the board meeting ("we're actually totally fine flying back ON the 19th") -- a plausible-sounding justification that makes the wrong date *more* convincing, not just silently wrong.
3. **No explicit re-check signal, anywhere.** Nothing in `instruction.md` or any agent-visible content (including Sam's Contacts note, which is flavor text only -- "Met at the Denver conference two years ago. Always down for a trip.") tells the agent to expect a change or hints that it should re-verify. The agent either independently values re-checking a stale conclusion before committing to something real, or it doesn't -- the task gives it no nudge either way.
4. **Budget-proximity trap.** The stale-date combo (return-19 flight \$220 + matching hotel \$90/night x 7 nights = \$630, total \$850) is *also* under the \$900 budget and cheaper than at least one correct-date combo -- so budget alone never disqualifies the wrong answer; only the corrected date does.
5. **Arithmetic trap.** Hotel price is listed per-night, not as a trip total -- the agent must multiply by (checkout - checkin) nights itself. One correct-date combo (ProAir flight \$410 + Grand Plaza \$110/night x 6 nights = \$660, total \$920) is *over* budget despite having the right dates -- date-correctness alone doesn't guarantee a pass either.
6. **Flight/hotel date-consistency trap.** Nothing stops an agent from picking a correctly-dated flight and an incorrectly-dated hotel (or vice versa) independently; `flight_hotel_dates_consistent` checks that both legs actually match each other, not just the answer key individually.
7. **Genuine re-verification, not a lucky race.** `genuine_not_premature` cross-references the booking's real confirm timestamp against `first_action_t + delay_seconds`. A correct-date booking that lands *before* the correction could possibly have been read is a lucky guess, not evidence of the capability -- and is scored accordingly (see Section 6 on the calibration history behind this check).
8. **No semantic shortcuts.** No accessibility tree, no app list, no "read screen as text" -- every one of the above requires actually looking at screenshots and finding real UI

elements (icons, drawer scroll position, native first-run dialogs) by pixel coordinates, the same way a hallucinated tap or a misread screenshot would fail on a real device.

5. Fairness audit against the assignment rubric

- **Solvable:** yes -- proven via two independent `harbor run -a oracle -y` passes (`overall: 1.0` both times), using the *same* tool scripts the agent uses, not a database shortcut. The oracle is a literal recorded human-style trace (see `solution/oracle_steps.py`).
- **Unambiguous:** the answer key encodes one true depart date, one true return date, one true budget ceiling -- all derivable from content that ships in the image. Nothing about grading depends on subjective reading.
- **Substantive / 3+ subgoals:** find true requirements (Calendar + Contacts + Chat, 3 sources) -> select a matching flight -> select a matching hotel -> review together -> confirm -> (implicitly) re-verify after the mid-mission update. More than 3 genuinely distinct subgoals.
- **Reproducible:** yes, with one disclosed caveat -- see Section 7 (ARM64 + host QEMU binfmt registration requirement).
- **Non-brittle:** the pre-correction read is not a "gotcha" -- it's the reasonable conclusion from real information, explicitly designed and documented that way (see `tests/answer_key.json`'s own notes field). Grading only fails an agent for *not re-checking after being told things changed*, not for a defensible initial read.
- **Real seed artifacts:** real Android content-provider rows (Calendar events, a Contacts card), a real SQLite catalog, a real Chat thread -- not synthetic placeholder text.
- **Hypothesis formation + verification:** the agent must form a requirements hypothesis from three sources, then a mid-mission event requires revising it -- structurally a verify-then-reconcile task, not a single-shot fetch.
- **Durable deliverable:** a confirmed TripDesk booking, persisted in SQLite, graded directly -- no separate report needed.
- **Net-new:** no scenario, app, asset, or task text is copied from MobileWorld, AndroidWorld, or any prior Collinear task -- confirmed by direct review before implementation began. Only high-level methodology (backend-DB verification, real device state, cross-app requirement) is shared, and that methodology is explicitly public/published, not proprietary to either project.

6. Calibration history (real evidence, not guesswork)

`delay_seconds` (how long after the agent's first action the corrected message arrives) was tuned against actual model behavior, not chosen arbitrarily:

Value	Evidence	Outcome
150s	Real trial, <code>claude-fable-5.1-high</code> , 66 actions	Booking confirmed at 493.5s -- the correction had <i>already</i> landed before the agent's first Chat read, so the trap was never exercised. Clean pass, but not evidence of the target capability.
150s	Real trial, <code>claude-fable-5.1-high</code> , 36 actions	Same outcome, confirmed at a later timestamp -- also never exercised the trap.
480s	Oracle re-validation (not a real-model trial)	Confirmed at 541.7s, <code>genuine_not_premature: 1.0</code> -- confirms 480s still leaves the oracle's own literal human-style pace enough room to solve it.

480s was chosen as just under the ~490-500s natural pace observed across two independent real-model completions, so a careful real agent is now likely to read Chat *before* the correction lands (seeing only the stale picture) and must deliberately return to Chat later to catch it -- rather than accidentally winning a race against its own latency. This is disclosed, not hidden: `tests/answer_key.json`'s `notes` field records this exact reasoning and the data points behind it.

Update: 4/4 completed real-model trials now exist against the 480s calibration, all via the Claude Code/OAuth harness after OpenRouter budget ran out (see Section 8) -- and all 4 produced the identical failure: `flight_id=1 + hotel_id=2`, return date 2026-10-19 (the stale, pre-correction date), `overall: 0.65` every time. A 5th OpenRouter attempt at 480s hit a genuine `402 Payment Required` mid-run (excluded from the failure-rate count as infra noise, not model evidence). See `RUN_REPORT.md` for the full results table and trace locations. The calibration fix worked exactly as intended: this is a real, reproducible, non-brittle failure, not a one-off.

Important scope caveat, stated plainly: 480s is calibrated to `claude-fable-5.1-high`'s own observed pace, not chosen to be unbeatable in general. The evidence this produces is that `claude-fable-5.1-high` fails a re-verification requirement timed to its own natural latency -- it is not evidence that this delay would defeat a materially faster or slower agent, and no claim to that effect is made here. See `REVIEWER_RESPONSE.md` point 1 for the full discussion.

7. Disclosed reproducibility caveats

- **ARM64-only.** The emulator binary (`android-emulator-linux-aarch64-dgx-spark`, unofficial build, v0.2.0-unofficial, SHA256 `aaa426635e9b760567931e98f2de260f6323d46855f54067eb1401061f80c265`) only runs on an ARM64 host with KVM. There is no x86_64 build in this task. This is a genuine special-resource requirement, not a bug -- documented plainly rather than silently assumed.
- **Host-global QEMU binfmt.** `adb` (Google's linux-amd64-only platform-tools) runs under the host's QEMU binfmt handler for cross-arch execution. This registration is host-global and can be lost across host reboots/session boundaries -- re-register via `docker run --rm --privileged tonistiigi/binfmt --install all` if `adb` calls start failing with an `exec-format` error. See `README.md`.

- **Vendored, not re-downloaded, emulator binary.** The unofficial emulator binary's own CDN (`release-assets.githubusercontent.com`) was unreachable at TCP-connect level from this build environment (confirmed via `curl -v`) -- the binary is vendored into the build context (`environment/emu.tar.zst`) with SHA256 verification in the Dockerfile, rather than fetched at build time. Provenance URL is recorded in the Dockerfile comment for independent re-verification.
- **Requires `--privileged + /dev/kvm`.** Set via a task-authored `environment/docker-compose.yaml` that Harbor merges onto its own base compose file for the `main` service -- confirmed via reading Harbor's own source (`MAIN_SERVICE_NAME = "main"`). `task.toml` 's own schema has no field for this; it is set at the compose layer.

7a. "Works on my machine" -- what actually makes it work, and how to

run it elsewhere

This was built and run on an NVIDIA DGX Spark (ARM64, `aarch64` , `6.17.0-1032-nvidia`), but nothing about the task is specific to that machine. What it actually needs is:

1. **An ARM64 (`aarch64`) Linux host.** The vendored emulator binary is an ARM64 build (see Section 7) -- it will not run on `x86_64` at all, with or without KVM. This is the one genuinely non-negotiable requirement.
2. **Hardware-accelerated virtualization exposed to Docker** -- i.e. `/dev/kvm` present and usable (`--privileged + --device /dev/kvm` , already wired in `environment/docker-compose.yaml`). Any ARM64 machine with nested-virtualization-capable hardware and KVM enabled works the same way: a bare-metal ARM64 server, an ARM64 cloud instance with KVM passthrough (e.g. AWS Graviton bare-metal, Oracle Cloud Ampere bare-metal, Google Axion bare-metal), or another ARM64 workstation. Nothing in the task talks to DGX-specific tooling (no CUDA, no GB10- specific driver, no NVIDIA-only APIs) -- the GPU-adjacent hardware on a DGX Spark is irrelevant here; only the CPU's virtualization extensions and KVM matter.
3. **Docker with QEMU binfmt registered for cross-arch exec** (see Section 7's binfmt caveat) -- a one-line, one-time host command (`docker run --rm --privileged tonistiigi/binfmt --install all`), not a DGX-specific step.

In short: **"emulator inside an emulator" is the actual requirement** -- any ARM64 machine that can itself run a hardware-accelerated Android emulator can run this task. Nothing here depends on this being *this* DGX Spark specifically, or even an NVIDIA machine at all.

Verified claim, not assumed: checked directly against Google's own SDK package feed (`https://dl.google.com/android/repository/repository2-3.xml`) before writing this -- the official Android Emulator has **never shipped a Linux `aarch64` build**. The feed lists exactly one Linux emulator archive per version (`emulator-linux_x64-*.zip`); macOS gets both `darwin_x64` and `darwin_aarch64` (Apple Silicon), Linux gets only `x64` . This means the

unofficial third-party ARM64 Linux build vendored here (Section 7) isn't a substitute for an official option Google happens to also offer -- there is no official option for this combination at all. Anyone reproducing this on ARM64 Linux is stuck accepting the same unofficial-binary trust decision this submission made, not something this task could route around by "just using the official build instead."

No single build is portable across both x86_64 and ARM64 hosts, as currently packaged.

This task's Dockerfile is ARM64-only (unofficial Linux ARM64 emulator + `arm64-v8a` system image). A genuinely portable version would need to detect host architecture at build time and branch: on `x86_64`, use Google's official `emulator-linux_x64` build with an official `x86_64` system image (a strictly better-supported path -- no unofficial binary needed at all there); on ARM64, keep what's here today. That dual-path Dockerfile is real, buildable work and has **not** been implemented in this submission -- claiming otherwise would be inaccurate. What's true today: this task runs on any ARM64+KVM Linux host, and nothing about it is specific to this particular DGX Spark; it does not yet run on `x86_64` hosts at all.

8. Economic viability -- a real, disclosed limitation

This is a long-horizon, image-heavy task by design (context accumulates across 40-70 turns of screenshots -- that accumulation *is* the capability under test), run at `reasoning_effort=high`. That combination is genuinely expensive to run per trial:

- One completed real trial (`claude-fable-5.1-high` , 150s calibration, 66 actions): ~**\$20.6** (OpenRouter credit went from \$32.54 to \$11.92 remaining on the same key).
- One partial trial that hit a `402 Payment Required` mid-run after only ~25 turns: ~**\$11.6** (credit went from \$11.92 remaining -- after a \$10 top-up to \$21.92 -- down to \$10.33), likely inflated by automatic reconnect retries against the same failing request.

Under a small fixed budget (e.g. \$35 total), this task cannot support multiple trials. At most one high-effort trial is affordable, and nothing guarantees it completes rather than hitting a transient infra failure like the one above. This is a direct, structural tension: the same design choice that makes the task a legitimate long-horizon/context-rot eval (accumulating screenshot history, high reasoning effort) is also what makes it expensive, and the two aren't separable without weakening the capability under test.

Disclosed conclusion, not papered over: under tight budget constraints, this task should be run and reported as an $n=1$ result (pass/fail on a single high-effort trial), not as statistically averaged evidence across repeated trials -- *if* the metered pay-per-token route is the only option available.

What actually resolved this: Harbor also ships a `claude-code` agent that authenticates against a Claude subscription's included usage (Pro/Max rate-limited window) instead of per-token billing (`claude setup-token` -> `CLAUDE_CODE_OAUTH_TOKEN` , `CLAUDE_FORCE_OAUTH=1`). Switching to it (after two small `task.toml` network-allowlist fixes -- see `RUN_REPORT.md`) let 4 real trials run at effectively zero marginal cost against an existing subscription. This is disclosed

as exactly what it is: appropriate for a single operator validating a task design against their own subscription, not a claim that this is how a third party evaluating this submission under a fixed metered budget would reproduce it cost-for-cost. Both numbers are reported honestly in RUN_REPORT.md: the real OpenRouter per-trial cost above, and the fact that the reported 4-trial result set was actually obtained via the subscription route once that budget ran out.